PROJECT MODEL, API, ANGULAR & SQL Integration Guide

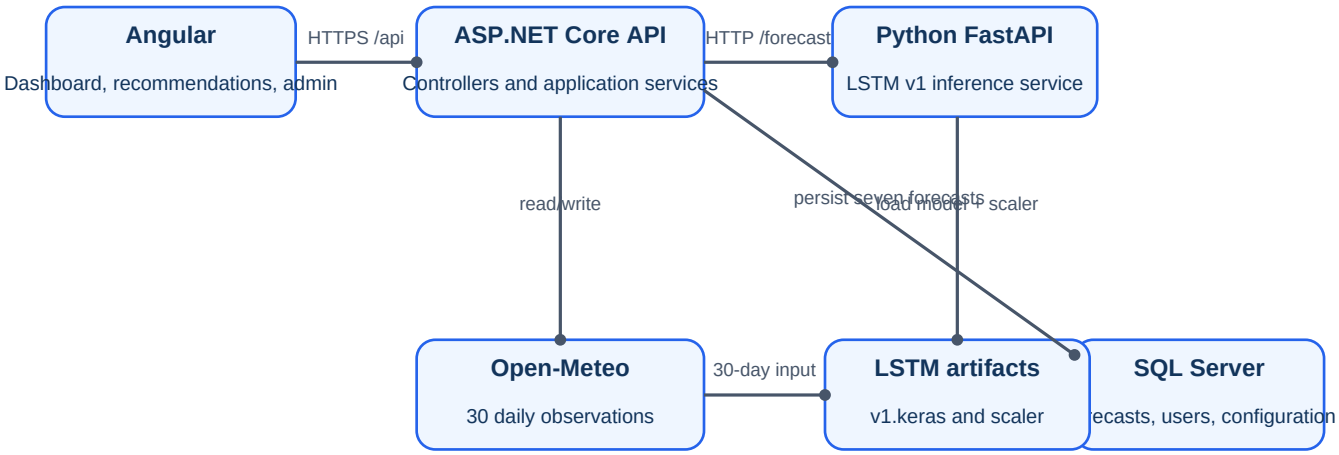
A beginner-friendly explanation of the working MSc prototype for Anuradhapura District

Purpose

This guide explains how the Angular web application, ASP.NET Core API, SQL Server database and Python LSTM forecasting service work together. It describes the implementation currently present in this repository; it does not expose connection strings, server addresses, user names, passwords or other private infrastructure details.

The project predicts seven days of rainfall, temperature and humidity. A separate explainable recommendation workflow uses the latest eligible stored forecast and clearly reports factors that are unavailable or only partially assessed.

1. Overall architecture



No database credentials travel to Angular or the Python service. SQL access remains inside ASP.NET Core.

The browser never communicates directly with SQL Server, Open-Meteo, or the Python model. Angular calls the ASP.NET Core API. The API owns authentication, data validation, SQL persistence and calls to the Python forecasting boundary.

2. The trained LSTM v1 model and Python FastAPI

The forecasting service is a Python FastAPI application. Its **WeatherForecastPredictor** loads the saved LSTM v1 model and its matching scaler. The API accepts exactly 30 consecutive daily observations containing rainfall, temperature and humidity, then returns seven dated predictions with the model version.

Part	Actual repository location	Responsibility
FastAPI endpoints	src/forecasting-service/forecasting_service/api.py	GET /health and POST /forecast
Inference predictor	forecasting_service/inference/predictor.py	Validates 30 observations and produces seven predictions
Saved artifacts	src/forecasting-service/artifacts/v1_era5_2010_2025/	v1.keras, scaler and evaluation metadata
Evaluation	forecasting_service/evaluation/metrics.py	MAE and RMSE calculations

The LSTM is a weather model, not a crop-selection model. Crop suitability is evaluated later by ASP.NET Core services using the persisted forecast and configuration/evidence that is available.

3. ASP.NET Core to Python communication

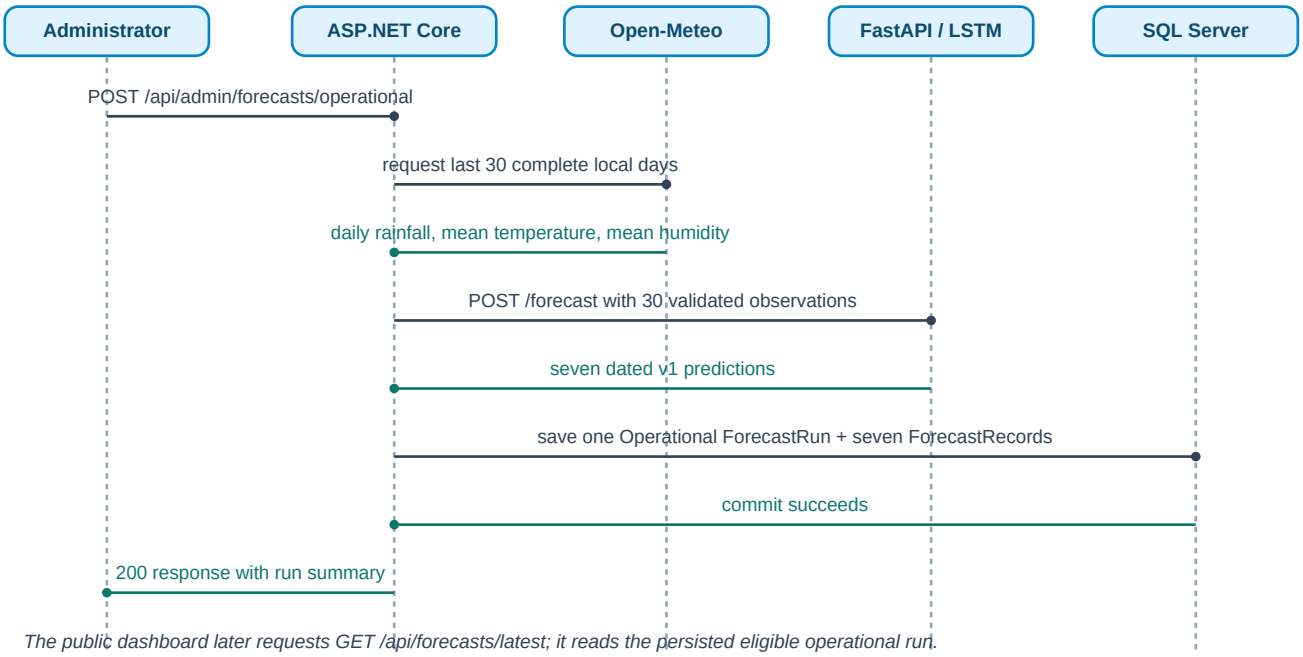
• DependencyInjection.cs registers an HTTP client for IWeatherForecastingClient.

• PythonForecastingClient.cs sends the validated observation payload to Python POST /forecast.

• ProviderForecastOrchestrationService.cs checks the observation window, calls Python, validates seven output days and coordinates persistence.

• The backend response retains model version and operational provenance rather than treating operational input as historical ERA5 evaluation data.

4. Fresh operational forecast flow



The Open-Meteo adapter used for the controlled operational path is **OpenMeteoForecastPastDaysWeatherSource**. It requests hourly observations, aggregates them into local daily values, and fails closed when the required 30 complete days are not available. The administrator-only trigger is **POST /api/admin/forecasts/operational**.

A successful run stores one parent ForecastRun marked Operational and seven child ForecastRecord rows. GET /api/forecasts/latest selects only a complete eligible operational run; it does not mix rows from different runs.

5. SQL Server persistence and retrieval

Database concern	Actual implementation
EF Core context	AnuradhapuraAiDbContext.cs in the Infrastructure project
Forecast persistence	ProviderForecastOrchestrationService.cs writes ForecastRun and ForecastRecord entities
Read path	ForecastService.cs and GET /api/forecasts/latest
Recommendation read path	RecommendationOrchestrationService.cs consumes the selected operational forecast
Connection safety	Development metadata is configuration; the password is held only in local user secrets

Connection concept: development configuration identifies a generic **<server>**, **<database>** and **<user>**; a local secret store supplies the password at runtime. The real values and the full connection string are intentionally excluded from this guide and from source control.

6. Angular to ASP.NET Core communication

Angular uses ApiService in **src/frontend/src/app/core/api.service.ts**. The service calls backend API routes through the local proxy during development. Components display backend results rather than connecting to SQL Server or Python directly.

User interface	Angular call	Backend route	What the user sees
Public dashboard	getLatestForecast()	GET /api/forecasts/latest	Latest persisted seven-day weather forecast and provenance
Recommendations	evaluateRecommendation()	POST /api/recommendations/evaluate	Forecast evidence, factor explanations and limitations
Administrator	generateOperationalForecast()	POST /api/admin/forecasts/operational	A new operational run summary
Authentication	login/register methods	POST /api/auth/login and /api/auth/register	Role-aware authenticated experience

The public dashboard component is public-dashboard.component.ts. It asks the API for the latest stored run and displays it only when it has seven daily values. recommendation-page.component.ts requests a read-only evaluation and presents missing factors explicitly, so a partial indicator is not presented as full scientific suitability.

7. End-to-end example

Step	Request or action	Response / result
1	Administrator selects Generate operational forecast	Angular POSTs /api/admin/forecasts/operational with the administrator token.
2	Backend gets observations	Open-Meteo adapter returns 30 complete daily values after validation and aggregation.
3	Backend calls model	Python POST /forecast returns seven v1 predictions.
4	Backend saves run	SQL stores one Operational ForecastRun and seven ForecastRecord rows atomically.
5	Public dashboard loads	GET /api/forecasts/latest returns the stored run, model version and seven days.
6	User requests recommendation	POST /api/recommendations/evaluate uses that same eligible run and returns evidence-aware crop results.

8. Source-file map and endpoints

Area	Key source files	Relevant endpoints
API setup	Api/Program.cs; Infrastructure/DependencyInjection.cs	/health
Forecast API	Api/Controllers/ForecastsController.cs; Infrastructure/Forecasting/ForecastService.cs	GET /api/forecasts/latest; POST /api/forecasts
Operational integration	AdminController.cs; ProviderForecastOrchestrationService.cs; OpenMeteoForecastPastDaysWeatherSource.cs; PythonForecastingClient.cs	POST /api/admin/forecasts/operational
Recommendation	RecommendationsController.cs; RecommendationOrchestrationService.cs	POST /api/recommendations/evaluate
Python model	forecasting_service/api.py; inference/predictor.py	GET /health; POST /forecast
Frontend	core/api.service.ts; public-dashboard.component.ts; recommendation-page.component.ts; admin-readiness.component.ts	Calls the API routes above

9. Run and verify the working application

• Keep the approved local development configuration and local user secret available. Do not place database passwords in appsettings files or source code.

• Start the Python service from src/forecasting-service: python -m uvicorn forecasting_service.api:app --host 127.0.0.1 --port 8000.

• Start the ASP.NET Core API from src/backend/AnuradhapuraAI.Api: dotnet run --urls http://127.0.0.1:5055.

• Start Angular from src/frontend: npm start -- --host 127.0.0.1 --port 4300.

• Verify /health on the API and Python service. Open the dashboard in the browser and confirm the latest forecast contains seven days.

• With an authorized Administrator account, use the Administration page to generate an operational forecast. Confirm the dashboard and recommendation page reference the same ForecastRun ID.

Important limits: the operational integration is technically working, but historical ERA5 evaluation metrics do not establish live Open-Meteo operational accuracy. Crop results remain evidence-aware and should not be presented as guaranteed agronomic outcomes.